

Rastera

A Symbolic Programming Language for Procedural Graphics

Group Members: Muhammad Hassan Abaid (23L-0584) Muneeb ur Rehman (23L-0511)
 Danyal Ahmed (23L-0746)

1. Purpose

Rastera is a small compiled programming language for **procedural graphics and generative art**. It combines a general programming core with a minimal raster-graphics runtime, allowing programs to implement normal algorithms (such as insertion sort and merge sort) as well as generate images, while remaining practical to implement within the project deadline.

The main design goal is to make the syntax visibly different from conventional C-like or Python-like languages. Rastera therefore represents major control-flow concepts with symbolic operators rather than words such as `function`, `for`, `while`, `if`, `else`, and `return`.

2. Novel Syntax and Core Language

Rastera uses square-bracket blocks and a compact set of control symbols. The first implementation will use a single scalar type, `i32`, plus mutable arrays of `i32`. Variables are introduced by assignment, which keeps the type system and semantic analysis manageable.

Rastera form	Meaning	Example
@name(args) []	Function definition	@max(a,b) [...]
i :: a..b []	Range loop (bounds inclusive)	* i :: 0..9 [...]
(cond) []	Condition-controlled loop	* (x < 100) [...]
? (cond) [] : []	If / else (condition in parentheses)	? (x > 0) [...] : [...]
^ value;	Return	^ x;
name := value;	Create variable / reassign	x := 10;
<1,2,3>	Array literal	a := <9,4,7,2>;
arr(n)	Zero-filled array of n elements	tmp := arr(n);

The grammar remains small enough to implement cleanly in Bison. Function calls and arithmetic retain familiar notation so the language stays readable while its control-flow syntax remains distinctive. Every condition is written in parentheses (for both `?` and `*`), which keeps the grammar free of ambiguity with array indexing such as `a[i]`.

3. Required Features

The planned compiler supports all required language constructs:

- **Variables:** signed 32-bit integer variables, created through `:=` on first assignment and reassigned by later `:=` in the same or an enclosing scope.
- **Arrays:** mutable integer arrays, array literals, allocation with `arr(n)`, indexing, and indexed assignment. Arrays are passed to functions by reference.
- **Functions:** user-defined functions, integer and mutable array parameters, calls, return values, and recursion (needed for merge sort).
- **Loops:** range loops (inclusive bounds) and condition-controlled loops.
- **If/else:** symbolic conditional syntax using `?` and `:`.
- **Expressions:** arithmetic, comparisons, logical operators `&` and `|`, and nested expressions. Logical operators are short-circuit, so `j >= 0 & a[j] > key` never reads `a[-1]`.

4. Algorithm Example: Insertion Sort

This Rastera program demonstrates a complete sorting algorithm using functions, arrays, variables, both loop forms, indexing, and assignments. Merge sort is supported through recursion and `arr(n)` for its temporary buffer.

```
@insertion(a[], n) [  
  * i :: 1..(n - 1) [  
    key := a[i];  
    j := i - 1;  
    * (j >= 0 & a[j] > key) [  
      a[j + 1] := a[j];  
      j := j - 1;  
    ]  
    a[j + 1] := key;  
  ]  
  ~ 0;  
]
```

5. Domain-Specific Graphics Features

Rastera adds a small set of built-in raster operations: `canvas`, `pixel`, `line`, `rect`, `rgb`, and `export`. Colors are packed 32-bit integers, avoiding a larger graphics type system. A small generative program can therefore be written as:

```
@main() [  
  canvas(256, 256);  
  * y :: 0..255 [  
    * x :: 0..255 [  
      shade := (x * x + y * y) % 255;  
      ? (shade > 128) [  
        pixel(x, y, rgb(255, 255, 255));  
      ] : [  
        pixel(x, y, rgb(20, 20, 20));  
      ]  
    ]  
  ]  
  export();  
  ~ 0;  
]
```

The compiled program writes the raster buffer to a simple PPM image, providing a visual demonstration without a large graphics framework.

6. Compiler Architecture

The compiler will use the required toolchain:

Rastera Source → Flex Lexer → Bison Parser → AST / Semantic Checks → LLVM IR → Native Executable

Flex will recognize symbolic tokens, identifiers, literals, operators, and punctuation. Bison will construct the AST. Semantic analysis will manage scopes, function signatures, arrays, and symbol lookup. The LLVM backend will lower expressions, branches (including short-circuit logic), loops, arrays, and calls to IR. The initial target is Ahead-of-Time compilation; JIT is optional.

7. Scope and Feasibility

To keep the project achievable for three members, the first version excludes classes, general strings, garbage collection, generics, and GPU programming. The graphics runtime will be a small C/C++ support library called from generated LLVM IR. Work can be divided into (1) Flex/Bison frontend, (2) AST, symbol table and semantic analysis, and (3) LLVM code generation plus graphics runtime. The core language is completed first and the graphics runtime is added last. The final demo will show both insertion sort and a generated image.